

Desarrollo de Aplicaciones Web Avanzado

Resumen consolidado del curso — Sesiones 01 a 15

Node.js • Express • Sequelize / Mongoose • JWT
Next.js 15 (App Router) • Tailwind CSS • NextAuth • SEO • Despliegue

Versión ampliada: incluye el **código real** de las diapositivas (capturas de editor, diagramas y tablas) que el resumen anterior, basado solo en texto extraído, había perdido.

Carrera: Diseño y Desarrollo de Software — C24 | 2026

Tabla de Contenidos

Sesión 01 — Introducción a Node.js	3
Sesión 02 — Gestión de Módulos (HTTP en Node.js)	5
Sesión 03 — Gestión de Archivos y Streams	7
Sesión 04 — Introducción a Express.js	9
Sesión 05 — APIs RESTful con Express.js	12
Sesión 06 — Conexión a una Base de Datos	14
Sesión 07 — Autenticación y Seguridad (Backend)	16
Sesión 08 — Despliegue y Buenas Prácticas	18
Sesión 09 — Introducción a Next.js	20
Sesión 10 — Enrutamiento y Páginas (Next.js)	21
Sesión 11 — Componentes y Estilización	23
Sesión 12 — API's Integradas (API Routes en Next.js)	27
Sesión 13 — Autenticación y Seguridad (NextAuth.js)	30
Sesión 14 — Optimización y SEO	32
Sesión 15 — Conexión Backend-Frontend	35
Síntesis para el Proyecto — Stack Completo	38
Índice de correcciones respecto al resumen anterior	41

Sesión 01 — Introducción a Node.js

Conceptos clave

- **Node.js** es un entorno de ejecución de JavaScript, multiplataforma, de código abierto y con una **arquitectura orientada a eventos**.
- Trabaja con un **único hilo de ejecución** (single thread) que organiza todo el flujo de trabajo.
- Está basado en el motor **V8 de Google** (el mismo de Chrome), diseñado para ejecutar JavaScript de forma extremadamente rápida.
- **Cuándo usarlo**: muchas operaciones de E/S (acceso a ficheros, base de datos, procesamiento de información) y cuando se necesita una conexión persistente cliente-servidor (juegos online, chats, herramientas de colaboración).
- **Características**: multiplataforma, asíncrono, ideal para bases de datos no relacionales, comunidad muy activa, ampliable con módulos vía **NPM (Node Package Manager)**, y permite front-end, back-end y móvil con un mismo lenguaje.

Ventajas vs. Desventajas

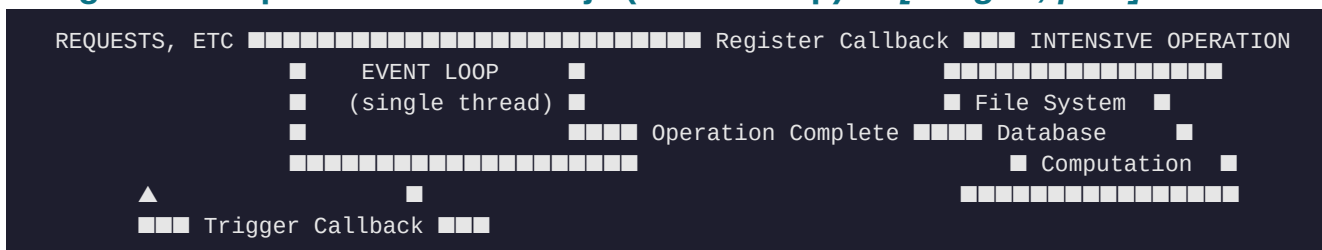
Ventajas	Desventajas
Eficiencia y escalabilidad (E/S sin bloqueo basada en eventos, alta concurrencia).	Uso intensivo de CPU: no es ideal para cálculos pesados o procesamiento de imágenes (un solo hilo puede bloquearse).
Unificación del lenguaje (JS en servidor y cliente).	Madurez de algunas bibliotecas menor que en otros ecosistemas.
Amplio ecosistema de módulos vía npm (>1 millón de paquetes).	Curva de aprendizaje del paradigma asíncrono.

Stack del curso — Arquitectura MERN/MENN

Flujo de datos:

- **Cliente (Next.js / React)**: envía peticiones HTTP (**fetch**/Axios) al backend.
- **Servidor Backend (Express + Node.js)**: procesa las peticiones y se comunica con MongoDB.
- **MongoDB (Base de datos)**: almacena y recupera datos.
- **Respuesta al Cliente**: Express devuelve la información a Next.js para renderizar la UI.

Diagrama: Arquitectura de Node.js (Event Loop) — [imagen, p.11]



Las peticiones entran al **Event Loop** (hilo único); las operaciones intensivas (sistema de archivos, base de datos, cómputo) se delegan y, al completarse, disparan su *callback* de vuelta al loop.

Código real de las diapositivas

Servidor HTTP "Hello World" con el módulo nativo `node:http` — [imagen editor, p.20]

```
const { createServer } = require('node:http');

const hostname = '127.0.0.1';
const port = 3000;
```

```
const server = createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

Instalación / verificación (opción Docker mostrada en nodejs.org) — *[imagen editor, p.19]*

```
# Descarga la imagen de Docker de Node.js:
docker pull node:22-alpine

# Crea un contenedor de Node.js e inicia una sesión shell:
docker run -it --rm --entrypoint sh node:22-alpine

# Verifica la versión de Node.js:
node -v # Debería mostrar "v22.18.0".

# Verifica versión de npm:
npm -v # Debería mostrar "10.9.3".
```

*La diapositiva marca los pasos de descarga en nodejs.org: (1) versión **v22.18.0 (LTS)**, (2) gestor **npm**, (3) SO Windows, (4) arquitectura x64, (5) **Windows Installer (.msi)**.*

Diagrama: Calendario de versiones LTS — *[imagen, p.21]*

Las versiones principales entran como **Current** (~6 meses); las **pares** pasan a **LTS Activo** y luego a **Mantenimiento** (soporte total ~30 meses). En producción usar solo **LTS Activo** o **LTS en Mantenimiento**. Estados mostrados: Main = UNSTABLE; Node.js 18/20 = MAINTENANCE; Node.js 22 = ACTIVE→MAINTENANCE; Node.js 23 = CURRENT; Node.js 24 = CURRENT→ACTIVE.

Conclusiones

- Node.js es ideal para aplicaciones a gran escala con múltiples peticiones concurrentes; su E/S no bloqueante de un solo hilo lo hace excelente para tiempo real y flujo de datos.
- Cuenta con la comunidad y el repositorio de paquetes de código abierto más grandes del mundo (NPM).

Sesión 02 — Gestión de Módulos (HTTP en Node.js)

Conceptos clave

- **HTTP (HyperText Transfer Protocol)**: protocolo base de la web para intercambiar información entre clientes y servidores. Modelo **cliente-servidor** y **sin estado (stateless)** — cada solicitud es independiente.
- **HTTP vs HTTPS**: HTTP envía datos en texto plano (menos seguro); **HTTPS usa TLS/SSL** para cifrar y garantizar autenticidad (login, pagos, APIs seguras).
- **Métodos HTTP más usados**:
 - **GET**: solicitar/leer datos (no modifica).
 - **POST**: enviar datos para **crear** un recurso.
 - **PUT**: **reemplazar** por completo un recurso existente.
 - **PATCH**: actualización **parcial** de campos.
 - **DELETE**: eliminar un recurso.
- **Request** (cliente → servidor): método + headers (Content-Type, Authorization) + body (JSON). **Response** (servidor → cliente): código de estado + headers + body (HTML, JSON, archivos).
- **JSON** (JavaScript Object Notation): formato ligero de intercambio de datos, alternativa más sencilla que XML.
- **REST clients** para probar APIs: **Postman** (gráfico), **cURL** (línea de comandos), **Insomnia**.

Código real de las diapositivas

Servidor web con enrutamiento manual usando el módulo `http` — *[imagen editor, p.13]*

```
// Importar el módulo http nativo de Node.js
const http = require("http");

// Definir el puerto del servidor
const PORT = 3000;

// Crear el servidor
const server = http.createServer((req, res) => {
  // Configurar cabecera de respuesta
  res.setHeader("Content-Type", "text/html; charset=utf-8");

  // Manejo básico de rutas
  if (req.url === "/") {
    res.statusCode = 200;
    res.end("<h1>Bienvenido al servidor Node.js ■</h1>");
  } else if (req.url === "/about") {
    res.statusCode = 200;
    res.end("<h1>Acerca de nosotros</h1><p>Este es un servidor básico.</p>");
  } else if (req.url === "/contact") {
    res.statusCode = 200;
    res.end("<h1>Contacto</h1><p>Escríbenos a contacto@ejemplo.com</p>");
  } else {
    res.statusCode = 404;
    res.end("<h1>404 - Página no encontrada</h1>");
  }
});

// Iniciar el servidor
server.listen(PORT, () => {
  console.log(`Servidor corriendo en http://localhost:${PORT}`);
});
```

Pruebas en Postman — colección DAWA02ApiRest sobre `http://localhost:4000/students` — [imágenes p.14–17]

La colección define 5 requests: GET ALL, GET BY ID, POST CREATE, PUT UPDATE, DEL DELETE.

GET `http://localhost:4000/students` → 200 OK

```
[
  { "id": 1, "name": "Ana", "grade": 18 },
  { "id": 2, "name": "Luis", "grade": 11 },
  { "id": 3, "name": "Pedro", "grade": 16 }
]
```

POST `http://localhost:4000/students` (body raw JSON) → 201 Created

```
// Request body
{ "name": "María", "grade": 20 }
// Response
{ "name": "María", "grade": 20, "id": 4 }
```

PUT `http://localhost:4000/students/2` (actualiza) → 200 OK

```
// Request body
{ "grade": 11 }
// Response
{ "id": 2, "name": "Luis", "grade": 11 }
```

DELETE `http://localhost:4000/students/3` (sin body) → 200 OK

```
{ "id": 3, "name": "Pedro", "grade": 16 }
```

Conclusiones

- El módulo `http` de Node.js permite crear servidores que reciben solicitudes y envían respuestas, manejando métodos y códigos HTTP.
- Gestionar comunicaciones HTTP con JSON sienta las bases para APIs y aplicaciones web modernas, escalables y seguras.

Sesión 03 — Gestión de Archivos y Streams

Conceptos clave

- **Streams:** colecciones de datos que se procesan **por partes**, permitiendo manejar grandes volúmenes sin cargarlos completos en memoria.
- **Tipos:** **Readable** (lectura), **Writable** (escritura), **Duplex** (ambos), **Transform** (modifican datos mientras fluyen).
- **Ventajas:** menor consumo de memoria, mayor eficiencia, procesamiento en tiempo real, mejor experiencia de usuario.
- **Módulo fs** para archivos. Métodos **síncronos** (bloquean), **asíncronos** (callbacks/promesas) y **asíncronos con streams** (máxima eficiencia).
 - Operaciones: `readFile()`, `writeFile()`, `copyFile()`, `unlink()`, `mkdir()`, `rmdir()` → `rm()`, `stat()`.
 - Comparación: `readFile()` carga todo en memoria; `createReadStream()` procesa por fragmentos (ideal para archivos grandes).
- **Backpressure:** ocurre cuando el destino procesa más lento que la fuente; puede causar fugas de memoria. Estrategias: verificar el resultado de `write()`, pausar con `.pause()`, ajustar `highWaterMark`.

Código real de las diapositivas

Streams legibles (Readable) — creación, eventos y modos

```
const readable = fs.createReadStream('archivo.txt');
// Configurable con opciones como el tamaño de fragmento (highWaterMark)
```

- **Eventos:** `'data'` (recibe fragmentos), `'end'` (finaliza lectura), `'error'` (captura problemas), `'close'` (cierra el stream).
- **Modos:** *Flujo* → emite `'data'` automáticamente; *Pausa* → requiere `readable.read()` explícito.

Streams escribibles (Writable) — creación, métodos y eventos

```
const writable = fs.createWriteStream('destino.txt');
```

- **Métodos clave:** `write()` (envía datos), `end()` (finaliza), `cork()/uncork()` (optimizan escrituras múltiples).
- **Eventos:** `'drain'` (buffer vacío), `'finish'` (todas las escrituras completas), `'error'` (fallos).

Tuberías (pipes) y encadenamiento — [imagen, p.9]

```
// Concepto pipe(): conecta un stream legible con uno escribible
source.pipe(destination);

// Encadenamiento: crea flujos de procesamiento complejos
source.pipe(transform1).pipe(transform2).pipe(destination);

// Ejemplo zlib: comprime archivos eficientemente
const gzip = zlib.createGzip();
readStream.pipe(gzip).pipe(writeStream);
```

Manejo de errores (imprescindible en todos los streams)

```
stream.on('error', (err) => console.error(err));
```

Casos de uso

- Procesamiento de archivos grandes (logs, datasets, backups) con memoria limitada.

- APIs y servidores web: transmisión de respuestas HTTP (FormData + fetch/axios), streaming de audio/video, descarga progresiva.
- ETL y procesamiento de datos: transformación de formatos, limpieza, análisis en tiempo real.

Conclusiones

- Los streams son cruciales para aplicaciones Node.js de alto rendimiento; permiten manejar grandes volúmenes con recursos limitados.
- **Mejores prácticas:** liberar recursos correctamente, manejar errores en cada stream, modularizar el código.

Sesión 04 — Introducción a Express.js

Conceptos clave

- **Express.js:** framework minimalista de Node.js que facilita la creación de servidores web y APIs. Ligero y rápido, con soporte de **middlewares**, sistema de **rutas** robusto y compatible con múltiples motores de plantillas.
- **Rutas:** mapeo entre una solicitud HTTP específica (URL + método) y la función que responde. Direccionan la solicitud al controlador adecuado.
- **Middleware:** funciones que se ejecutan **antes** de que la solicitud llegue a su ruta final; se encolan y sirven para validar datos, autenticar, registrar (logging), manipular encabezados, etc. Se agregan con `app.use()` o por verbo HTTP.
- **Enrutamiento modular** con la clase `express.Router()` para dividir rutas en varios archivos.
- **Entry point:** archivo `app.js` o `index.js`, punto de inicio donde se configuran app, middlewares y puerto.

Código real de las diapositivas

Ruta básica GET — *[imagen editor, p.7]*

```
const express = require("express");
const app = express();

// Ruta GET
app.get("/users", (req, res) => {
  res.send("Lista de usuarios");
});
```

Ruta con parámetro dinámico :id — *[imagen editor, p.8]*

```
// Ruta GET
app.get("/users", (req, res) => {
  res.send("Lista de usuarios");
});

// Ruta con parámetro
app.get("/users/:id", (req, res) => {
  res.send(`Usuario con ID: ${req.params.id}`);
});
```

Una solicitud a `/users/123` devolverá "Usuario con ID: 123".

Enrutamiento modular con `express.Router()` — *[imagen editor, p.9]*

```
const express = require("express");
const app = express();

// Crear un router
const userRouter = express.Router();

// Definir rutas dentro del router
userRouter.get("/", (req, res) => {
  res.send("Lista de usuarios");
});

userRouter.get("/:id", (req, res) => {
  res.send(`Detalle del usuario con ID: ${req.params.id}`);
});

// Usar el router bajo el prefijo /users
app.use("/users", userRouter);
```

```
app.listen(3000, () => {
  console.log("Servidor corriendo en http://localhost:3000");
});
```

Middleware de registro (logger) — [imágenes editor, p.10 y p.13]

```
// Middleware básico: registra la hora y la ruta de cada request
function logger(req, res, next) {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next(); // pasa al siguiente middleware o ruta
}

// Usar el middleware en toda la app
app.use(logger);
```

Diagrama (p.10): Request → [Middleware → App] → Response. El middleware envuelve a la app.

Middleware de manejo de errores (debe ir al final) — [imagen editor, p.11]

```
// Middleware de manejo de errores (debe ir al final)
app.use((err, req, res, next) => {
  console.error(err.stack); // Lo imprime en consola
  res.status(500).json({
    message: "Ocurrió un error en el servidor",
    error: err.message, // en producción a veces se oculta
  });
});
```

Si un middleware recibe **4 argumentos** (**err, req, res, next**), Express lo interpreta como middleware de manejo de errores. Diagrama "Middleware Pattern" (Express JS): HTTP Request → Matched Route → CORS Middleware → CSRF Middleware → Auth Middleware → Main Task → HTTP Response.

Configuración básica completa de una app Express — [imagen editor, p.14]

```
const express = require("express");
const app = express();

// Middleware básico: registra la hora y la ruta de cada request
function logger(req, res, next) {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next(); // pasa al siguiente middleware o ruta
}

// Usar el middleware en toda la app
app.use(logger);

app.get("/", (req, res) => {
  res.send("¡Hola, mundo!");
});

app.get("/about", (req, res) => {
  res.send("Acerca de nosotros");
});

app.listen(3000, () => {
  console.log("Servidor corriendo en http://localhost:3000");
});
```

Express Generator (comandos reales de terminal) — [imágenes p.15–18]

```
# Paso 1: Instalar el generador globalmente
npm install -g express-generator

# Paso 2: Crear el proyecto (con motor de plantillas Pug)
express --view=pug myapp
```

```
# Paso 3: Instalar dependencias
cd my_project
npm install

# Paso 4: Ejecutar el proyecto (disponible en http://localhost:3000/)
npm start
```

Estructura generada (entry point) — *[imagen árbol de archivos, p.18]*

```
Base/
  ├── middlewares/
  │   └── logger.js
  ├── routes/
  │   └── users.routes.js
  ├── app.js
  ├── package-lock.json
  └── package.json
```

¿Por qué usar Express.js?

Simplicidad, rápido y ligero, extensible (ecosistema de middlewares), flexible, comunidad amplia, compatible con Node.js/npm, e ideal para APIs REST.

Conclusiones

- Las rutas son el puente entre las solicitudes y la lógica de negocio.
- Los middlewares aportan modularidad y reutilización, separando responsabilidades.
- Express Generator produce una estructura clara que favorece la escalabilidad.

Sesión 05 — APIs RESTful con Express.js

Conceptos clave

- **API (Application Programming Interface):** conjunto de reglas y protocolos que permite que aplicaciones se comuniquen; actúa como puente entre sistemas para intercambiar datos y funcionalidades.
- **API RESTful:** interfaz que sigue los principios **REST** (Representational State Transfer): usa métodos HTTP estándar, se basa en **recursos** accesibles por URLs, es **stateless**, y usa JSON/XML.
- **Principios REST:** Cliente-Servidor (separación front/back), Stateless (cada solicitud es autocontenida), Caché, Interfaz uniforme (URLs + métodos HTTP).
- **Mapeo de recursos (API de usuarios):**

Recurso	Método	URL
Obtener todos	GET	/usuarios
Obtener uno	GET	/usuarios/:id
Crear	POST	/usuarios
Actualizar	PUT	/usuarios/:id
Eliminar	DELETE	/usuarios/:id

- **Códigos de estado:** 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Internal Server Error.
- **Buenas prácticas:** JWT para proteger endpoints, CORS para controlar acceso entre dominios, documentar con Swagger/Postman.

Código real de las diapositivas

Archivo `server.js` — configuración base — *[imagen editor, p.11]*

```
const express = require("express");

const app = express();
const PORT = 3000;

// Middleware para trabajar con JSON
app.use(express.json());

// Iniciar servidor
app.listen(PORT, () => {
  console.log(`Servidor escuchando en http://localhost:${PORT}`);
});
```

CREATE — POST /usuarios — *[imagen editor, p.12]*

```
// Simulación de base de datos en memoria
let usuarios = [
  { id: 1, nombre: "Juan" },
  { id: 2, nombre: "María" }
];

// CREATE - Crear un usuario
app.post("/usuarios", (req, res) => {
  const nuevoUsuario = {
    id: usuarios.length + 1,
    nombre: req.body.nombre
  };
  usuarios.push(nuevoUsuario);
  res.status(201).json(nuevoUsuario);
});
```

```
});
```

READ — GET (todos y por ID) — *[imagen editor, p.13]*

```
// READ - Obtener todos los usuarios
app.get("/usuarios", (req, res) => {
  res.json(usuarios);
});

// READ - Obtener un usuario por ID
app.get("/usuarios/:id", (req, res) => {
  const usuario = usuarios.find(u => u.id === parseInt(req.params.id));
  if (!usuario) {
    return res.status(404).json({ mensaje: "Usuario no encontrado" });
  }
  res.json(usuario);
});
```

UPDATE (PUT) y DELETE — *[imagen editor, p.14]*

```
// UPDATE - Actualizar un usuario por ID
app.put("/usuarios/:id", (req, res) => {
  const usuario = usuarios.find(u => u.id === parseInt(req.params.id));
  if (!usuario) {
    return res.status(404).json({ mensaje: "Usuario no encontrado" });
  }
  usuario.nombre = req.body.nombre || usuario.nombre;
  res.json(usuario);
});

// DELETE - Eliminar un usuario por ID
app.delete("/usuarios/:id", (req, res) => {
  const usuarioIndex = usuarios.findIndex(u => u.id === parseInt(req.params.id));
  if (usuarioIndex === -1) {
    return res.status(404).json({ mensaje: "Usuario no encontrado" });
  }
  const usuarioEliminado = usuarios.splice(usuarioIndex, 1);
  res.json(usuarioEliminado[0]);
});
```

Conclusiones

- Express facilita la creación de endpoints RESTful de forma rápida y sencilla.
- REST aporta una estructura clara y estandarizada; la combinación permite servicios escalables, mantenibles y reutilizables, ideales para conectar web, móvil e IoT con el backend.

Sesión 06 — Conexión a una Base de Datos

Conceptos clave

- **Base de datos:** conjunto organizado de información que se almacena y consulta. Puede ser **Relacional (SQL)** — datos en tablas — o **No Relacional (NoSQL)** — documentos, clave-valor, grafos.
- **Relacionales (SQL):** tablas con filas/columnas, lenguaje SQL, relaciones con JOINS, transacciones ACID. Ej.: MySQL, PostgreSQL, SQL Server, Oracle. Escalabilidad **vertical**.
- **No Relacionales (NoSQL):** esquema flexible, escalabilidad **horizontal**, alto rendimiento. Ej.: MongoDB (documentos), Redis (clave-valor), Neo4j (grafos), Cassandra.
- **ORM (Object Relational Mapping):** mapea objetos del lenguaje a tablas relacionales, evitando escribir SQL directo. En Node.js: **Sequelize**, TypeORM, Prisma.
- **ODM (Object Document Mapping):** equivalente para NoSQL basado en documentos. En Node.js: **Mongoose** (el más popular), Typegoose, Waterline.

Característica	Relacional (SQL)	No Relacional (NoSQL)
Estructura	Tablas (filas/columnas)	Documentos, clave-valor, grafos
Esquema	Fijo y definido	Flexible y dinámico
Lenguaje	SQL	Varía (JSON, API, etc.)
Relaciones	JOINS	Generalmente sin JOINS directos
Escalabilidad	Vertical	Horizontal
Casos de uso	Finanzas, ERP, sistemas críticos	Big Data, IoT, apps web, redes sociales

Código real de las diapositivas

Configuración de conexión con Sequelize (SQL) — *[imagen editor, p.12]*

```
const { Sequelize } = require("sequelize");

const sequelize = new Sequelize("mi_base", "usuario", "password", {
  host: "localhost",
  dialect: "mysql"
});

sequelize.authenticate()
  .then(() => console.log("Conexión OK"))
  .catch(err => console.error("Error:", err));
```

Sequelize soporta MySQL, PostgreSQL, SQL Server y SQLite.

Definir un modelo con Sequelize — *[imagen editor, p.13]*

```
const { DataTypes } = require("sequelize");
const sequelize = require("../db"); // conexión configurada

// Definir modelo
const User = sequelize.define("User", {
  name: {
    type: DataTypes.STRING,
    allowNull: false
  },
  email: {
    type: DataTypes.STRING,
    unique: true
  },
  age: {
    type: DataTypes.INTEGER
```

```

    }
  });

module.exports = User;

```

Los atributos son columnas con su tipo (**STRING**, **INTEGER**...). **allowNull** y **unique** controlan restricciones. Sequelize genera la tabla automáticamente si no existe.

Instalación de Mongoose (NoSQL / MongoDB) — [imagen terminal, p.14]

```
npm install mongoose
```

Requisitos: Node.js + npm, y MongoDB corriendo — local (**mongodb://localhost:27017**) o en la nube con **MongoDB Atlas** (URI de conexión).

Configuración de conexión con Mongoose — [imagen editor, p.15]

```

const mongoose = require("mongoose");

mongoose.connect("mongodb://localhost:27017/mi_base", {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log("Conexión a MongoDB establecida"))
.catch(err => console.error("Error de conexión:", err));

```

■ ■ La diapositiva original muestra dos erratas tipográficas: **console.erro** (debería ser **console.error**) y **"Error de conexión"**. Aquí se corrigen a la forma correcta.

Definir un modelo con Mongoose (Schema + model) — [imagen editor, p.16]

```

const mongoose = require("mongoose");

// Definir esquema
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, unique: true },
  age: Number
});

// Crear modelo
const User = mongoose.model("User", userSchema);

module.exports = User;

```

type, **required**, **unique** establecen reglas para los documentos. El modelo (**mongoose.model**) permite interactuar con la colección; los documentos se manejan como objetos JSON.

Buenas prácticas (Node.js + BD)

- Definir esquemas claros con validaciones.
- Evitar consultas sin filtros (riesgo de alto consumo); usar **índices** para consultas frecuentes.
- Usar **variables de entorno** para credenciales (no hardcodear).
- Manejo de errores con **try/catch** y validaciones.

Conclusiones

- Node.js se conecta fácilmente a SQL y NoSQL; MongoDB encaja bien gracias a JSON.
- ORMs y ODMs simplifican el trabajo con BD. La elección depende de: consistencia (SQL) vs flexibilidad/escalabilidad (NoSQL).

Sesión 07 — Autenticación y Seguridad (Backend)

Conceptos clave

- **Autenticación:** valida que el usuario es quien dice ser ("¿Quién eres?"). Credenciales usuario/contraseña + factores adicionales (huella, códigos, autenticadores).
- **Autorización:** ocurre después; define qué recursos/acciones puede realizar el usuario ("¿Qué tienes permitido hacer?").
- **Métodos populares:** usuario/contraseña (con hashing seguro), **Tokens (JWT, OAuth)**, OAuth 2.0 / OpenID Connect (Google, Facebook, GitHub), SSO (Single Sign-On).
- **JWT (JSON Web Token):** estándar abierto (RFC 7519) para transmitir información entre partes como objeto JSON firmado. Evita almacenar sesiones en el servidor; verificación rápida y distribuida; firmado digitalmente (no manipulable sin detección).
- **Estructura de un JWT** (3 partes separadas por `.`):
 - **Header:** tipo de token y algoritmo de firma (ej. `HS256`).
 - **Payload:** claims/datos (`sub`: usuarioid, `iat`: emisión, `exp`: expiración).
 - **Signature:** firma de header+payload con clave secreta o par de claves.
- **Ventajas JWT:** escalabilidad (sin sesiones en servidor, ideal para microservicios), compatibilidad (web/móvil/APIs), autocontenido, rendimiento (solo verifica firma), seguridad (expiración, RS256).
- **CORS (Cross-Origin Resource Sharing):** mecanismo del navegador que controla qué dominios pueden hacer peticiones a tu API. Por defecto se bloquean solicitudes cross-origin. En Express se habilita con el paquete `cors`.

LocalStorage vs SessionStorage vs Cookies

Característica	LocalStorage	SessionStorage	Cookies
Persistencia	Hasta borrar manualmente	Hasta cerrar pestaña/navegador	Según expiración
Alcance	Todas las pestañas del dominio	Solo la pestaña actual	Todas las requests HTTP
Tamaño	~5–10 MB	~5 MB	~4 KB
Acceso JS	Sí	Sí	Solo si no es HttpOnly
Se envía al servidor	No	No	Sí
Seguridad	Riesgo XSS	Riesgo XSS	Más seguro (HttpOnly, Secure)

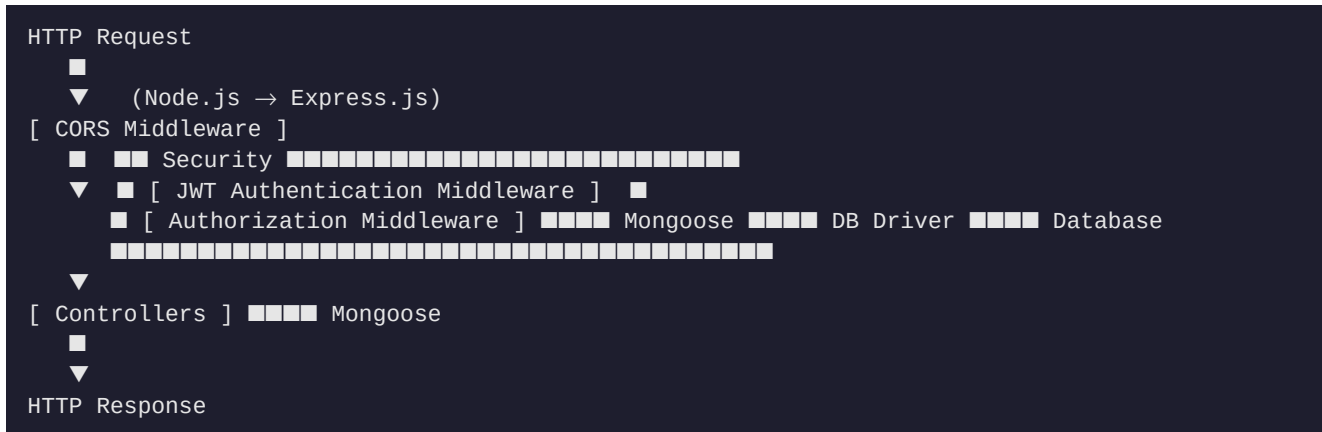
Diagrama: Flujo de autenticación JWT — [imagen, p.12]

```
USER REGISTRATION
Client → Server: (1) POST api/auth/signup { username, email, role, password }
Server:          (2) Check existing · Save User to database
Server → Client: (3) return Message ("Registered successfully!")

USER LOGIN
Client → Server: (1) POST api/auth/signin { username, password }
Server:          (2) Authenticate { username, password } · Create JWT string with a secret
Server → Client: (3) return { token, user info, authorities }

ACCESS RESOURCE
Client → Server: (4) Request data with JWT on x-access-token Header
Server:          (5) Check JWT Signature · Get user info & authenticate · Authorize using user's Authorities
Server → Client: (6) return Content based on Authorities
```

Diagrama: Arquitectura Node.js/Express con auth — [imagen, p.14]



El request atraviesa en orden: CORS → autenticación JWT → autorización → controladores. Mongoose/DB Driver conectan con la base de datos.

Consideraciones de seguridad y buenas prácticas

- Usar siempre **HTTPS**.
- Tiempos de **expiración cortos** (**exp**) e implementar **Refresh Tokens**.
- No almacenar datos sensibles en el payload del JWT.
- Mecanismo para invalidar tokens comprometidos.
- **No guardar tokens en LocalStorage**; preferir cookies seguras (HttpOnly, Secure).

Conclusiones

- JWT es un estándar moderno, flexible y escalable (sin sesiones en servidor).
- Requiere configuración de seguridad adicional (CORS, almacenamiento seguro, expiración) para ser robusto.

*Nota: esta sesión es conceptual/diagramática — no incluye capturas de código de editor; el paquete recomendado es **jsonwebtoken** (npm).*

Sesión 08 — Despliegue y Buenas Prácticas

Conceptos clave

- **Despliegue (deployment):** proceso por el cual una app pasa del entorno local a uno donde los usuarios pueden acceder. Incluye construir el código, configurar dependencias, ajustar variables de entorno y publicar en un servidor/nube.
- **Etapas comunes:** 1) Desarrollo, 2) Pruebas (Staging), 3) Producción, 4) Monitoreo (métricas, logs), 5) Actualización (mejoras/parches).
- **Tipos de despliegue:**
 - **Manual:** directo, simple pero propenso a errores.
 - **Automatizado:** scripts/pipelines repetibles.
 - **Blue-Green:** dos entornos idénticos; uno recibe tráfico mientras el otro se actualiza.
 - **Canary:** nueva versión a un pequeño grupo antes del despliegue completo.
 - **Rolling:** reemplazo progresivo de instancias sin interrumpir el servicio.
- **Variables de entorno:** configuran valores sensibles (claves, rutas, URLs) sin modificar el código.
- **Logs:** registros de eventos para auditar, depurar y medir. Herramientas: **Winston, Morgan** (Node), o servicios externos **Datadog, New Relic, Elastic Stack (ELK)**.
- **CI/CD:** Integración Continua (integrar y probar cambios frecuentemente, detectar errores temprano) + Entrega Continua (automatizar entrega/despliegue). Herramientas: **GitHub Actions**, Azure DevOps, Jenkins, GitLab CI/CD.
- **Render:** plataforma cloud que simplifica el despliegue: conectas GitHub, configuras variables de entorno y despliega automáticamente. Ventajas: integración con GitHub, despliegue tras cada push, SSL gratis, plan gratuito, logs en tiempo real.

Código real de las diapositivas

Archivo `.env` — variables de entorno de ejemplo — *[imagen terminal, p.10]*

```
PORT=3000
MONGODB_URI=mongodb://localhost:27017/auth_db
JWT_SECRET=f14e6a1c9843c52190c07232dfb9c0e467d5a910
JWT_EXPIRES_IN=1h
BCRYPT_SALT_ROUNDS=10
```

GitHub Actions — workflow — *[texto de diapositiva, p.13–14]*

GitHub Actions automatiza tareas mediante **workflows** definidos en un archivo YAML dentro del repositorio:

```
.github/workflows/deploy.yml
```

Se ejecuta ante eventos como `push`, `pull request` o `merge`. Puede: ejecutar pruebas automáticas, construir la aplicación y desplegarla en servicios cloud (Render, AWS, Vercel).

Buenas prácticas

- Usar entornos separados (dev, qa, staging, prod).
- No incluir claves/contraseñas en el código.
- Implementar CI/CD para minimizar errores humanos.
- Documentar el proceso de despliegue.
- Monitorear rendimiento y logs.

- Control de versiones limpio; probar en staging antes de producción.

Conclusiones

- El despliegue es una fase crítica: el código se convierte en un servicio real.
- La automatización con CI/CD es hoy una necesidad. Render + GitHub Actions permiten un flujo ágil y confiable.
- El objetivo final no es "subir código", sino **entregar valor de forma continua, segura y eficiente**.

Sesión 09 — Introducción a Next.js

Conceptos clave

- **Next.js:** framework de **React** de código abierto que amplía sus capacidades para crear aplicaciones web más rápidas, eficientes y mejor posicionadas en buscadores (SEO). Combina frontend y backend en un mismo entorno.
- **Problema que resuelve:** React es excelente en el cliente pero carece de SSR por defecto. Next.js integra **SSR** y **SSG**.
- **Historia:** creado por **Guillermo Rauch (CEO de Vercel)** en 2016. Evolución:
 - Next.js 1–5: enrutamiento automático + SSR básico.
 - Next.js 9: **API Routes** e **ISR**.
 - Next.js 13–14: **App Router** y **React Server Components**.
 - Next.js 15: **Turbopack** estable y compatibilidad con **React 19**.
- **Tipos de renderizado:**
 - **SSR (Server-Side Rendering):** genera el HTML en el servidor → mejor SEO.
 - **SSG (Static Site Generation):** páginas estáticas en compilación → máxima velocidad.
 - **ISR (Incremental Static Regeneration):** actualiza páginas estáticas sin reconstruir todo el sitio.
 - **CSR:** renderizado en cliente.
- **Características:** enrutamiento automático basado en archivos (`/pages` o `/app`), API Routes, optimización nativa (`next/image`, fuentes, CSS Modules, TypeScript), división automática de código, Turbopack.
- **Novedades Next.js 15 (2024):** React 19, Turbopack estable (reemplaza a Webpack), Server Actions más rápidas, nuevos hooks/APIs para rutas y datos.
- **Integración con React 19:** rendering concurrente, separación cliente-servidor (más seguridad), bundles optimizados (menos JS).

Código real de las diapositivas

Configuración del entorno de desarrollo

- Node.js **versión 18 o superior**.
- Gestor de paquetes **npm** o **yarn**.
- Editor recomendado: **VS Code** con extensiones de React y ESLint.

Creación de un proyecto básico — *[imagen terminal, p.16]*

```
npx create-next-app@latest mi-proyecto
cd mi-proyecto
npm run dev
```

■■ La diapositiva original muestra `npm rundev` (sin espacio). El comando correcto es `npm run dev`.

Conclusiones

- Next.js potencia a React con SSR, SSG e híbrido, mejorando rendimiento y SEO.
- Simplifica el desarrollo fullstack (páginas, APIs y componentes en un solo entorno).
- Su enfoque en optimización lo hace ideal para proyectos escalables; empresas como TikTok, Netflix, Uber y GitHub Docs lo usan.

Sesión 10 — Enrutamiento y Páginas (Next.js)

Conceptos clave

- **Next.js 15** (lanzado 21/oct/2024): soporte para React 19, consolida el **App Router**, incluye **Turbopack** estable (arranques hasta 77% más rápidos) y optimiza `fetch()` con nuevo sistema de caché y revalidación.
- **App Router vs Pages Router:**

	App Router (/app)	Pages Router (/pages)
Modelo	Moderno, basado en React Server Components	Tradicional, sistema de archivos simple
Ejecución	Divide claramente servidor/cliente	Cada archivo = una ruta (páginas completas)
Rendimiento	Mejor (transmisión de UI, carga parcial, menos JS)	Menor (~40% más lento)
Uso recomendado	Nuevos proyectos	Solo proyectos legados/migración

- **Coexistencia:** Next.js 15 permite usar ambos enfoques simultáneamente (migración gradual).

Estructura de carpetas en Next.js 15 (App Router)

```
/app
  layout.tsx      # Layout compartido
  page.tsx        # Página (ruta)
  loading.tsx     # Estado de carga
  error.tsx       # Manejo de errores
  blog/
    [slug]/      # Carpeta anidada = ruta dinámica
      page.tsx
```

- **Archivos especiales:** `layout.tsx`, `page.tsx`, `loading.tsx`, `error.tsx`.

Rutas dinámicas y grupos de rutas

```
[id]           → parámetro simple           /blog/123
[...catchAll] → captura múltiples segmentos /docs/a/b/c
[...optional] → parámetro opcional (existe o no)
(name)         → grupo de rutas: organiza sin afectar la URL
@dashboard     → rutas paralelas (varios slots en la misma vista)
(..)           → interceptación de rutas (modales/vistas paralelas)
```

Data Fetching en Next.js 15

Pages Router (clásico)

```
// SSG: se ejecuta en build
export async function getStaticProps() { /* ... */ }
export async function getStaticPaths() { /* ... */ }

// SSR: en cada solicitud
export async function getServerSideProps() { /* ... */ }
```

App Router (moderno)

```
// SSG con revalidación (ISR) dentro del componente/layout
export const revalidate = 60; // segundos

// Genera rutas estáticas
export async function generateStaticParams() { /* ... */ }
```

```
// SSR: sin caché
await fetch(url, { cache: 'no-store' });
// o
export const dynamic = 'force-dynamic';

// SSG forzado
await fetch(url, { cache: 'force-cache' }); // force-static
```

Comparativa (Pages Router vs App Router)

Concepto	Pages Router	App Router
Funciones principales	<code>getStaticProps</code> , <code>getServerSideProps</code> , <code>getStaticPaths</code>	<code>fetch()</code> , <code>generateStaticParams()</code> , <code>revalidate</code> , <code>dynamic</code>
Ejecución	Next.js llama explícitamente a las funciones exportadas	Next.js detecta exportaciones y las ejecuta automáticamente
Data fetching	Dentro de funciones de datos	Directamente en componentes (Server Components)
Pre-render SSG	<code>getStaticProps()</code> + <code>getStaticPaths()</code>	<code>generateStaticParams()</code> + <code>fetch()</code> con <code>force-static</code>
SSR	<code>getServerSideProps()</code>	<code>fetch()</code> con <code>{ cache: 'no-store' }</code> o <code>dynamic='force-dynamic'</code>
ISR	<code>revalidate</code> dentro de <code>getStaticProps()</code>	<code>export const revalidate =</code> <code><tiempo></code>
React Server Components	No compatible	Totalmente compatible

Conclusiones

- El **App Router** aprovecha el renderizado en servidor nativo, mejora el rendimiento y reduce el JS enviado al cliente.
- **Migración gradual** recomendada (empezar por rutas menos críticas).
- **Compatibilidad retroactiva:** Next.js 15 mantiene `getStaticProps` y `getServerSideProps`.
- **Fetch API** mejorada: controla caché, revalidación y renderizado sin funciones adicionales.

Sesión 11 — Componentes y Estilización

Conceptos clave

- **Componentes React:** bloques constructores de la UI (funcionales o de clase). Reciben datos vía **props**, gestionan su **estado** interno y son **reutilizables**.
- **CSS Modules:** estilos con **alcance local** (`.module.css`), sin conflictos de nombres, sintaxis CSS familiar. Ventaja: control total. Desventaja: más verboso, sin sistema de diseño integrado.
- **Tailwind CSS:** framework **utility-first**, clases pequeñas reutilizables, sistema de diseño integrado. Ventaja: desarrollo ultrarrápido, responsive simple (`sm:`, `md:`, `lg:`), bundle optimizado. Desventaja: HTML "saturado" de clases, curva de aprendizaje.
- **Combinación recomendada:** Tailwind (~80%: layout, spacing, responsive) + CSS Modules (~20%: animaciones complejas, estilos específicos).

Anatomía de un componente (código real, reconstruido de p.6–p.11)

Las diapositivas descomponen un componente en 11 partes. A continuación, el código completo en orden:

```
// 1. DIRECTIVA (Solo si es Client Component) – debe ir en la primera línea
"use client"

// 2. IMPORTACIONES
// Librerías externas
import { useState, useEffect } from 'react';
import Image from 'next/image';
import Link from 'next/link';

// Componentes locales
import Button from './Button';
import Card from './Card';

// Estilos
import styles from './Component.module.css';

// Utilidades y tipos
import { formatDate } from '@utils/helpers';
import type { User } from '@types';

// 3. INTERFACES/TYPES (TypeScript)
interface ComponentProps {
  title: string;
  user: User;
  onSubmit?: (data: string) => void; // ? = opcional
  isActive?: boolean;
}

// 4. DEFINICIÓN DEL COMPONENTE
export default function MyComponent({
  title,
  user,
  onSubmit,
  isActive = false // valor por defecto
}: ComponentProps) {

  // 5. HOOKS Y ESTADO
  const [count, setCount] = useState(0);
  const [data, setData] = useState(null);
  const [isLoading, setIsLoading] = useState(false);

  // 6. EFECTOS
```

```

useEffect(() => {
  // Lógica que se ejecuta al montar/actualizar
  console.log('Componente montado');
  // Cleanup function
  return () => {
    console.log('Componente desmontado');
  };
}, [count]); // Dependencias

// 7. MÉTODOS/FUNCIONES AUXILIARES
const handleClick = () => {
  setCount(count + 1);
  console.log('Click handled');
};

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  setIsLoading(true);
  try {
    // Lógica async
    const result = await fetchData();
    setData(result);
    onSubmit?.(result); // Llamar prop opcional
  } catch (error) {
    console.error(error);
  } finally {
    setIsLoading(false);
  }
};

const fetchData = async () => {
  const response = await fetch('/api/data');
  return response.json();
};

// 8. VARIABLES DERIVADAS
const doubleCount = count * 2;
const isEven = count % 2 === 0;
const statusClass = isActive ? 'active' : 'inactive';

// 9. RENDERIZADO CONDICIONAL TEMPRANO (Early Returns)
if (isLoading) {
  return <div>Cargando...</div>;
}
if (!user) {
  return <div>No hay usuario</div>;
}

// 10. RETURN - JSX (LA PARTE VISUAL)
return (
  <div className={styles.container}>
    <main className={styles.main}>{/* Contenido principal */}
      {user && ( /* Renderizado condicional inline */}
        <div className={styles.userInfo}>
          <Image src={user.avatar} alt={user.name} width={50} height={50} />
          <p>{user.name}</p>
        </div>
      )}

      <div className={styles.stats}> {/* Mostrar datos */}
        <p>Count: {count}</p>
        <p>Double: {doubleCount}</p>
        <p>Es par: {isEven ? 'Sí' : 'No'}</p>
      </div>
    </main>
  </div>
);

```

```

    </div>

    <button {/* Eventos */}
      onClick={handleClick}
      onMouseEnter={() => console.log('hover')}
      disabled={isLoading}
      className={styles.button}
    >
      Incrementar
    </button>

    <Card {/* Componentes hijos */}
      title="Tarjeta anidada" onClick={() => console.log('Card clicked')}
    >
      <p>Contenido children</p>
    </Card>
  </main>
</div>
);
}

// 11. EXPORTACIONES ADICIONALES (opcional)
export { type ComponentProps };
export const COMPONENT_VERSION = '1.0.0';

```

Eventos comunes y patrones de props — *[imagen editor, p.15]*

```

// Eventos de Mouse
onClick={() => {}}
onDoubleClick={() => {}}
onMouseEnter={() => {}}
onMouseLeave={() => {}}

// Eventos de Formulario
onChange={(e) => {}}
onSubmit={(e) => {}}
onFocus={() => {}}
onBlur={() => {}}

// Eventos de Teclado
onKeyDown={(e) => {}}
onKeyUp={(e) => {}}
onKeyPress={(e) => {}}

// Props básicas
<Component title="Hola" count={5} />

// Props con funciones (callbacks)
<Component onSave={(data) => console.log(data)} />

// Props con objetos
<Component user={{ name: 'Juan', age: 25 }} />

// Props con children
<Component>
  <p>Contenido hijo</p>
</Component>

// Spread de props
const props = { title: 'Hola', count: 5 };
<Component {...props} />

```

Integración de TailwindCSS en Next.js 15 — *[texto p.20]*

```
# 1. Instalar Tailwind y generar configuración
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p      # genera tailwind.config.js y postcss.config.js

# 2. Configurar rutas de contenido en tailwind.config.js
#   (indicar dónde buscar las clases)

# 3. Agregar directivas en app/globals.css:
#   @tailwind base; @tailwind components; @tailwind utilities;

# 4. Usar clases utilitarias y ejecutar:
npm run dev
```

Buenas prácticas al crear componentes

- Un componente, una responsabilidad; extraer lógica compleja a funciones separadas.
- Validar props con TypeScript o PropTypes; keys únicas en listas (no usar `index`).
- Evitar lógica compleja en el JSX; no hacer mutaciones directas del estado.
- No olvidar dependencias en `useEffect`; no abusar de `useEffect`.

Conclusiones

- No existe solución única: la elección CSS Modules vs Tailwind depende del proyecto/equipo.
- La combinación de ambos es poderosa. Next.js + Tailwind se configura en minutos. Prioriza **consistencia sobre perfección**.

Sesión 12 — API's Integradas (API Routes en Next.js)

Conceptos clave

- **API Routes:** permiten crear endpoints backend **dentro de la app Next.js**, sin servidor separado. Cada archivo en `app/api` responde a peticiones HTTP; enfoque **serverless**.
- **Dónde/cómo se definen:** en App Router el archivo debe llamarse `route.ts` o `route.js`. Puedes exportar una función por método HTTP (`GET`, `POST`, `PUT`, `DELETE`, `OPTIONS`, `HEAD`).
- **Internamente:** cada `route.ts` es un **Route Handler**. Usa la **API estándar de Fetch** (`Request/Response`), **no** `req/res` de Express. Se ejecuta del lado del servidor.
- **Tipos de rutas:** estáticas (`/api/status`), dinámicas (`[id]`), anidadas (`/api/users/[id]/posts/[postId]`), catch-all (`[...slug]`).
- **Cuándo usarlas:** para lógica backend dentro de Next.js (esquema monolítico), exponer endpoints al frontend, evitar un Express separado. **Evítalas** si ya tienes un backend dedicado, o si solo necesitas datos internos para Server Components (usa `fetch()` directo).
- *"Empieza con un monolito bien hecho, no con microservicios mal hechos."*

Código real de las diapositivas

Ubicación y ruta básica GET — [imagen, p.6]

```
app/
├── api/
│   └── users/
│       └── route.ts
export async function GET(request: Request) {
  return new Response(JSON.stringify({ message: "Hola desde Next.js" }), {
    status: 200,
    headers: { "Content-Type": "application/json" }
  });
}
```

Acceso a datos con repositorio (GET + POST) — [imagen, p.8]

```
import { getUsers, createUser } from '@lib/userRepository'

export async function GET() {
  try {
    const users = await getUsers()
    return Response.json(users, { status: 200 })
  } catch (error) {
    console.error(error)
    return Response.json({ error: 'Error al obtener usuarios' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    const body = await request.json()
    if (!body.name || !body.email) {
      return Response.json({ error: 'Datos incompletos' }, { status: 400 })
    }
    const user = await createUser(body)
    return Response.json(user, { status: 201 })
  } catch (error) {
    console.error(error)
    return Response.json({ error: 'Error al crear usuario' }, { status: 500 })
  }
}
```

```
}
```

Caché/revalidación y cookies/headers — *[imagen, p.9]*

```
// Caché, dinámica y revalidación (Next.js 15 no cachea endpoints por defecto)
export const revalidate = 60; // Revalida cada 60 segundos
export const dynamic = "force-static"; // Cachea la respuesta
export const dynamic = "force-dynamic"; // Siempre genera desde el servidor

// Cookies y Headers
import { cookies, headers } from "next/headers";

export async function GET() {
  const token = cookies().get("session");
  const referer = headers().get("referer");
  return new Response(`Token: ${token?.value}, Referer: ${referer}`);
}
```

CORS y seguridad (headers manuales) — *[imagen, p.10]*

```
export async function GET() {
  return new Response("OK", {
    headers: {
      "Access-Control-Allow-Origin": "*",
      "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
      "Access-Control-Allow-Headers": "Content-Type, Authorization",
    },
  });
}
```

Parámetros dinámicos `[id]` (Path Params) — *[imagen, p.11]*

```
app/
  api/
    users/
      [id]/
        route.ts

// api/users/123
export async function GET(
  request: Request,
  { params }: { params: { id: string } }
) {
  return new Response(`El ID recibido es ${params.id}`);
}
```

Query Params (searchParams) — *[imagen, p.12]*

```
// api/users?id=123&active=true
export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const id = searchParams.get("id");
  const active = searchParams.get("active");
  return new Response(`ID=${id}, activo=${active}`);
}
```

Buenas prácticas

- Retornar siempre un status code claro (400, 401, 404, 500).
- Definir tipos con TypeScript.
- Centralizar el acceso a datos (`@/lib/db.ts` o repositorios).
- Para API pública, agregar control de CORS.

- Evitar `new Response(JSON.stringify(...))`; preferir `Response.json()`.

Conclusiones

- Las API Routes integran el backend directamente en Next.js sin servidor externo, con endpoints organizados y escalables en `app/api`.
- Su enfoque serverless mejora el despliegue y son una alternativa práctica a Express para proyectos fullstack.

Sesión 13 — Autenticación y Seguridad (NextAuth.js)

Conceptos clave

- **Métodos de autenticación en Next.js:**
 - **Credenciales:** usuario ingresa correo/contraseña y el servidor los valida.
 - **OAuth (Google, GitHub, etc.):** delega la autenticación a un servicio externo.
 - **Tokens (JWT):** para comunicación entre servicios o API externa.
 - **Cookies:** el servidor valida la sesión sin exponer datos sensibles al cliente.
- **NextAuth.js:** librería que facilita la autenticación en Next.js sin construir la infraestructura desde cero. Incluye proveedores OAuth, sesiones vía cookies, JWT y callbacks. Compatible con **App Router**; adaptadores para PostgreSQL, MongoDB, etc.

Código real de las diapositivas

Instalación y configuración (CredentialsProvider) — *[imagen editor, p.6]*

```
npm install next-auth

// app/api/auth/[...nextauth]/route.ts
import CredentialsProvider from "next-auth/providers/credentials";
// ...
providers: [
  CredentialsProvider({
    name: "Credentials",
    credentials: {
      username: { label: "Username", type: "text", placeholder: "jsmith" },
      password: { label: "Password", type: "password" }
    },
    async authorize(credentials, req) {
      const user = { id: "1", name: "J Smith", email: "jsmith@example.com" }
      if (user) {
        return user
      } else {
        return null
      }
    }
  })
]
// ...
```

Esta configuración crea automáticamente los endpoints para iniciar sesión, cerrar sesión e inspeccionar la sesión activa.

Inicio de sesión desde el cliente (signIn) — *[imagen editor, p.7]*

```
"use client";

import { signIn } from "next-auth/react";

export default function Login() {
  async function handleLogin() {
    await signIn("credentials", { email: "admin@test.com", password: "123456" });
  }

  return (
    <button onClick={handleLogin}>
      Iniciar sesión
    </button>
  )
}
```

```
);  
}
```

Manejo de sesiones y cookies (Server Component + .env) — [imagen editor, p.8]

```
import { auth } from "@auth";  
  
export default async function Dashboard() {  
  const session = await auth();  
  return <div>Bienvenido {session?.user?.name}</div>;  
}  
  
# .env  
NEXTAUTH_SECRET=your_secret_here  
NEXTAUTH_URL=http://localhost:3000
```

NextAuth genera cookies seguras que almacenan la referencia a la sesión o al token JWT. Desde el servidor se lee con `auth()` o `getServerSession()`.

Protección de rutas — [imagen editor, p.9]

```
// Opción 1: Middleware (protege rutas antes de permitir acceso)  
export { default } from "next-auth/middleware"  
  
export const config = { matcher: ["/dashboard"] }  
  
// Opción 2: Manualmente en un Server Component  
import { redirect } from "next/navigation";  
import { auth } from "@auth";  
  
const Page = async () => {  
  const session = await auth();  
  if (!session) redirect("/login");  
  return <div>Privado</div>;  
};
```

Validaciones de seguridad

- Configurar cookies como `httpOnly` y `secure` (impide acceso por JavaScript).
- Ejecutar siempre bajo **HTTPS** en producción.
- Para JWT, usar una **clave robusta** definida en variables de entorno (la diapositiva muestra un ejemplo de clave segura; se recomienda generarla con una herramienta como `openssl rand -base64 32`).
- Implementar **hashing de contraseñas** y bloqueo temporal tras varios intentos fallidos.

Buenas prácticas

Sí: mantener claves/secretos en variables de entorno; evitar exponer información sensible en el token; implementar roles y permisos en callbacks de sesión; revisar logs para auditorías. **No:** guardar contraseñas en texto plano; no validar entradas con Provider de credenciales; no usar HTTPS en producción; no limitar intentos de login.

Conclusiones

- NextAuth.js es una solución completa y flexible: combina OAuth, credenciales y JWT sin flujos manuales complejos.
- Su integración nativa con App Router y el manejo automático de sesiones/cookies facilitan la protección de rutas y el control de accesos.

Sesión 14 — Optimización y SEO

Conceptos clave

- **La optimización en Next.js** es estructural: la estrategia de renderizado, las imágenes y el tamaño del bundle impactan la experiencia, la conversión y la capacidad de Google de comprender el contenido.
- **Arquitectura de ejecución:** Next.js corre en servidor, cliente, edge o pre-render estático. SSR impacta el tiempo de respuesta; SSG elimina latencia; ISR combina rapidez + actualización; la segmentación del bundle reduce el trabajo del navegador.
- **Estrategias de rendimiento:** fragmentación del servidor, React Server Components (menos JS al cliente), carga diferida de módulos, minimizar re-renderizados (memoización), optimización de fuentes (fonts locales para evitar FOIT/FOUT).
- **SEO en Next.js:** produce HTML accesible para el crawler desde el primer byte (a diferencia de las SPA). Se basa en rutas predecibles, contenido semántico, metadatos correctos y páginas rastreables.
- **Herramientas de monitoreo:** Lighthouse, Web Vitals, PageSpeed Insights, Google Search Console.

Código real de las diapositivas

Estrategia de renderizado (ISR) — *[imagen editor, p.5]*

```
export const revalidate = 30;
export const dynamic = "force-static";
// Esto obliga a que una ruta de alta lectura sea pre-generada y actualizada cada 30 segundos
```

Optimización de fuentes (fonts locales) — *[imagen editor, p.6]*

```
import localFont from "next/font/local";

const Inter = localFont({
  src: "./Inter.woff2",
  display: "swap"
});
```

Optimización de imágenes con `next/image` — *[imagen editor, p.7]*

```
import Image from 'next/image';

export default function Profile() {
  return (
    <Image
      src="/images/avatar.png"
      alt="User profile picture"
      width={180}
      height={180}
      priority={false}
    />
  );
}
```

`Image` automatiza optimización, resizing y **lazy loading** por defecto; sirve formatos modernos y mejora métricas como **LCP**.

Meta tags dinámicos (`generateMetadata`) — *[imagen editor, p.9]*

```
export async function generateMetadata({ params }) {
  const data = await fetch(`https://api.myapp.com/blog/${params.slug}`).then(r => r.json());

  return {
    title: data.title,
    description: data.excerpt,
```

```

    openGraph: {
      url: `https://mysite.com/blog/${params.slug}`,
      type: "article",
      images: [data.cover]
    }
  };
}

```

Garantiza títulos únicos, descripciones coherentes y metadatos para redes sociales (Open Graph).

Rich Snippets con JSON-LD — *[imagen editor, p.10]*

```

export default function ProductPage() {
  const schema = {
    "@context": "https://schema.org",
    "@type": "Product",
    "name": "iPhone 15 Pro",
    "offers": {
      "@type": "Offer",
      "price": "999",
      "priceCurrency": "USD"
    }
  }

  return (
    <>
      <script
        type="application/ld+json"
        dangerouslySetInnerHTML={{
          __html: JSON.stringify(schema)
        }}
      />
      <h1>iPhone 15 Pro</h1>
      <p>Precio: $999</p>
    </>
  );
}

```

Sitemap XML (estructura) — *[imagen editor, p.11]*

```

<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://acme.com</loc>
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
    <changefreq>yearly</changefreq>
    <priority>1</priority>
  </url>
  <url>
    <loc>https://acme.com/about</loc>
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
    <changefreq>monthly</changefreq>
    <priority>0.8</priority>
  </url>
  <url>
    <loc>https://acme.com/blog</loc>
    <lastmod>2023-04-06T15:02:24.021Z</lastmod>
    <changefreq>weekly</changefreq>
    <priority>0.5</priority>
  </url>
</urlset>

```

Sitemap dinámico (generado con datos reales) — *[imagen editor, p.12]*

```

export async function GET() {
  const pages = await fetch("https://api.myapp.com/pages").then(r => r.json());
}

```

```

const xml = pages
  .map(p => `
    <url>
      <loc>https://mysite.com/${p.slug}</loc>
      <lastmod>${p.updatedAt}</lastmod>
    </url>
  `)
  .join("");

return new Response(
  `<?xml version="1.0" encoding="UTF-8"?>
    <urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">${xml}</urlset>`,
  { headers: { "Content-Type": "application/xml" } }
);
}

```

robots.txt — [\[imagen editor, p.13\]](#)

```

User-agent: *
Disallow: /admin/
Disallow: /api/
Allow: /

Sitemap: https://mysite.com/sitemap.xml

```

Comunica qué áreas rastrear y dónde está el sitemap; evita que Google pierda tiempo en paneles internos, pruebas, APIs o contenido duplicado.

Google Search Console

Permite ver cómo Google interpreta el sitio: páginas indexadas y por qué, consultas/clics/impresiones/CTR por URL, errores de indexación, problemas móviles y velocidad real. Útil para desarrolladores, equipos de contenido, PM/líderes y QA.

Conclusiones

- La optimización y el SEO en Next.js requieren un enfoque integral: renderizado híbrido, metadatos dinámicos, datos estructurados, optimización de imágenes y sitemaps dinámicos.
- Clave: combinar estas técnicas, **medir consistentemente** y ajustar de forma continua.

Sesión 15 — Conexión Backend-Frontend

Conceptos clave

- **Arquitectura de la solución:**
 - **Backend (Express):** API RESTful en puerto **3001**; datos, lógica de negocio, autenticación, middlewares.
 - **Frontend (Next.js):** app en puerto **3000**; Server + Client Components; rutas API opcionales para **BFF** (Backend for Frontend).
 - **Comunicación:** HTTP/HTTPS con JSON.
- **Stack:** Express (APIs REST) + Next.js (React híbrido SSR/SSG/CSR) + TypeScript (tipado estático). Ventajas: full-stack en JS/TS, buena DX, performance + SEO, escalabilidad.
- **fetch (Next.js) vs axios:**
 - **fetch:** integrado con caché/revalidación, soporte nativo en Server Components, menor bundle; API más verbosa para configuraciones complejas.
 - **axios:** interceptors para tokens/errores, mejor para Client Components con lógica compleja, sintaxis más limpia; dependencia externa (~13 kb).

Código real de las diapositivas

Tipos compartidos entre backend y frontend — *[imagen editor, p.8]*

```
// types/user.ts (shared)
export interface User {
  id: string;
  name: string;
  email: string;
  createdAt: Date;
}

export interface ApiResponse<T> {
  success: boolean;
  data?: T;
  error?: string;
}
```

Ventaja: un solo lugar para definir los contratos de datos, reduciendo inconsistencias.

fetch de Next.js en un Server Component — *[imagen editor, p.6]*

```
// app/users/page.tsx
async function getUsers() {
  const res = await fetch('http://localhost:3001/api/users', {
    next: { revalidate: 60 } // Cache por 60 segundos
  });
  return res.json();
}

export default async function UsersPage() {
  const users = await getUsers();
  return <UserList users={users} />
}
```

axios (instancia + uso en Client Component) — *[imagen editor, p.7]*

```
// lib/api.ts
import axios from 'axios';

export const api = axios.create({
```

```

    baseUrl: process.env.NEXT_PUBLIC_API_URL,
    headers: { 'Content-Type': 'application/json' }
  });
// components/UserForm.tsx
'use client';
const handleSubmit = async (data: User) => {
  try {
    await api.post('/users', data);
  } catch (error) {
    handleError(error);
  }
};

```

Setup del backend (Express + CORS) — *[imagen editor, p.9]*

```

// backend/src/server.ts
import express from 'express';
import cors from 'cors';

const app = express();
app.use(cors({ origin: 'http://localhost:3000' }));
app.use(express.json());

app.get('/api/users', (req, res) => {
  res.json({ success: true, data: users });
});

app.listen(3001);

```

Manejo de errores consistente (backend + frontend) — *[imagen editor, p.11]*

```

// backend middleware
app.use((err, req, res, next) => {
  res.status(err.status || 500).json({
    success: false,
    error: err.message
  });
});

// frontend utility
export async function fetchWithErrorHandling(url: string) {
  const res = await fetch(url);
  if (!res.ok) {
    throw new Error(`Error ${res.status}: ${res.statusText}`);
  }
  return res.json();
}

```

Validación de datos con Zod (backend) — *[imagen editor, p.14]*

```

// Backend con Zod
import { z } from 'zod';

const userSchema = z.object({
  email: z.string().email(),
  name: z.string().min(2)
});

app.post('/api/users', (req, res) => {
  const result = userSchema.safeParse(req.body);
  if (!result.success) {
    return res.status(400).json({ error: result.error });
  }
  // Procesar datos validados

```

```
});
```

Buenas prácticas

- **Variables de entorno:** usar `.env.local` (nunca commitear), separar públicas (`NEXT_PUBLIC_*`) de privadas, validar con schema (Zod/Joi). Nunca exponer API keys en el cliente.
- **Autenticación:** JWT + httpOnly cookies + refresh tokens; axios interceptors para tokens; middleware de Next.js para proteger rutas; rate limiting, CORS y CSRF.
- **Performance:** Backend → caché (Redis), paginación, compression. Frontend → Server Components, React Query/SWR, image optimization, lazy loading.
- **Herramientas:** Zod (validación), Prisma/Mongoose (ORM/ODM), React Query (estado servidor), Swagger/OpenAPI (docs), ESLint + Prettier, Husky (git hooks).

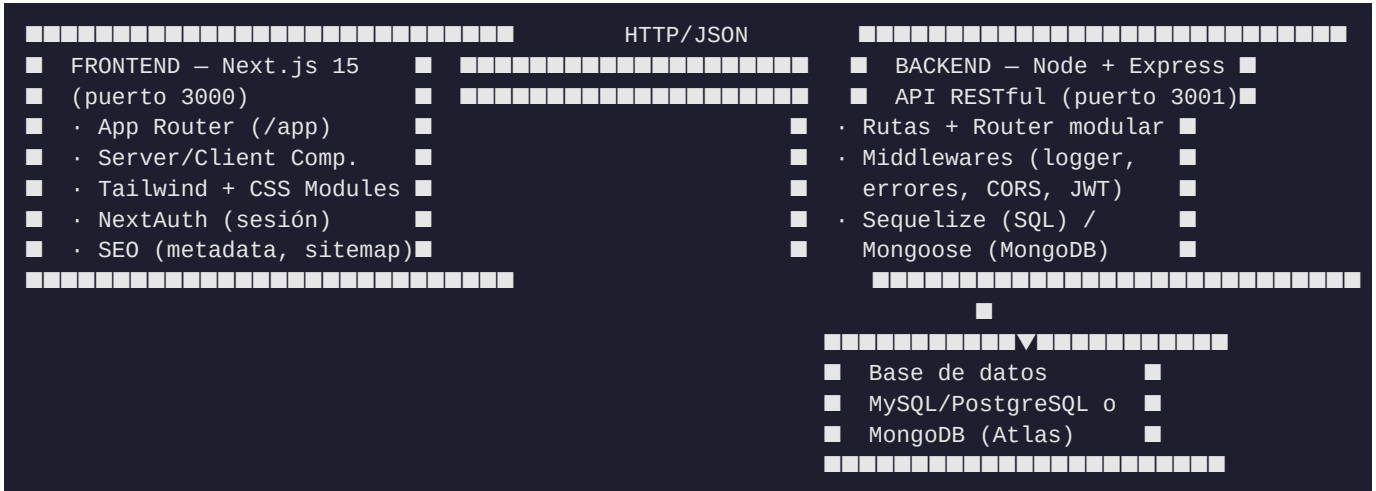
Conclusiones

- **Type safety end-to-end** con TypeScript; desarrollo ágil con hot reload en ambos lados; ecosistema JavaScript unificado.
- Excelente performance y SEO con Next.js + flexibilidad de Express para cualquier lógica backend.

Síntesis para el Proyecto — Stack Completo

Esta sección combina todo el stack del curso en un flujo coherente para construir una aplicación fullstack: **Node.js + Express + Sequelize/Mongoose + JWT** en el backend, y **Next.js (App Router) + Tailwind + NextAuth + SEO** en el frontend, con despliegue automatizado.

Visión de la arquitectura



1. Backend — Node.js + Express + ORM/ODM + JWT

Servidor base con middlewares

```
const express = require("express");
const cors = require("cors");
const app = express();

app.use(cors({ origin: "http://localhost:3000" })); // permitir el frontend
app.use(express.json()); // parsear JSON

// Middleware de logging (S04)
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next();
});
```

Modelo de datos (elegir uno según la BD)

```
// Opción SQL – Sequelize (S06)
const User = sequelize.define("User", {
  name: { type: DataTypes.STRING, allowNull: false },
  email: { type: DataTypes.STRING, unique: true },
});

// Opción NoSQL – Mongoose (S06)
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, unique: true },
});
const User = mongoose.model("User", userSchema);
```

Endpoints CRUD RESTful (S05) + validación (Zod, S15)

```
const { z } = require("zod");
const userSchema = z.object({ email: z.string().email(), name: z.string().min(2) });

app.post("/api/users", async (req, res) => {
```

```

    const result = userSchema.safeParse(req.body);
    if (!result.success) return res.status(400).json({ error: result.error });
    const user = await createUser(result.data);
    res.status(201).json(user);
  });

app.get("/api/users", async (req, res) => res.json(await getUsers()));
app.get("/api/users/:id", async (req, res) => { /* find by id, 404 si no existe */ });
app.put("/api/users/:id", async (req, res) => { /* actualizar */ });
app.delete("/api/users/:id", async (req, res) => { /* eliminar */ });

```

Autenticación JWT (S07) + manejo de errores (S15)

```

const jwt = require("jsonwebtoken");

// Login: firmar token
const token = jwt.sign({ sub: user.id, role: user.role }, process.env.JWT_SECRET, { expiresIn: "1h" });

// Middleware de protección
function auth(req, res, next) {
  const token = req.headers.authorization?.split(" ")[1];
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ error: "No autorizado" });
  }
}

// Middleware de errores (siempre al final)
app.use((err, req, res, next) => {
  res.status(err.status || 500).json({ success: false, error: err.message });
});

app.listen(3001);

```

2. Frontend — Next.js App Router + Tailwind + NextAuth

Consumo del backend desde un Server Component (S15)

```

// app/users/page.tsx
async function getUsers() {
  const res = await fetch("http://localhost:3001/api/users", { next: { revalidate: 60 } });
  return res.json();
}
export default async function UsersPage() {
  const { data: users } = await getUsers();
  return <UserList users={users} />;
}

```

Autenticación con NextAuth (S13)

```

// Cliente: iniciar sesión
"use client";
import { signIn } from "next-auth/react";
// await signIn("credentials", { email, password });

// Servidor: leer sesión y proteger
import { auth } from "@/auth";
export default async function Dashboard() {
  const session = await auth();
  if (!session) redirect("/login");
  return <div>Bienvenido {session.user?.name}</div>;
}

// Proteger rutas por middleware
export { default } from "next-auth/middleware";

```

```
export const config = { matcher: ["/dashboard"] };
```

Estilización con Tailwind (S11)

```
export default function Card({ title }: { title: string }) {
  return (
    <div className="rounded-lg shadow-md p-4 md:p-6 bg-white hover:shadow-lg transition">
      <h2 className="text-xl font-bold text-gray-800">{title}</h2>
    </div>
  );
}
```

SEO — metadata dinámica + sitemap (S14)

```
export async function generateMetadata({ params }) {
  const data = await fetch(`https://api.myapp.com/blog/${params.slug}`).then(r => r.json());
  return {
    title: data.title,
    description: data.excerpt,
    openGraph: { url: `https://mysite.com/blog/${params.slug}`, type: "article", images: [data.cover] },
  };
}
```

Complementar con: [next/image](#) (imágenes optimizadas), **JSON-LD** (rich snippets), **sitemap dinámico** y **robots.txt**.

3. Configuración de entorno (.env)

```
# Backend
PORT=3001
MONGODB_URI=mongodb://localhost:27017/app_db      # o cadena SQL
JWT_SECRET=<clave_robusta>
JWT_EXPIRES_IN=1h
BCRYPT_SALT_ROUNDS=10

# Frontend (Next.js)
NEXT_PUBLIC_API_URL=http://localhost:3001
NEXTAUTH_SECRET=<clave_robusta>
NEXTAUTH_URL=http://localhost:3000
```

4. Despliegue (S08)

- **CI/CD con GitHub Actions:** workflow en [.github/workflows/deploy.yml](#), disparado por [push/pull_request](#).
- **Render / Vercel:** conectar repositorio GitHub, configurar variables de entorno, despliegue automático tras cada push, SSL gratuito.
- Backend (Express) → Render/Railway; Frontend (Next.js) → Vercel.

Checklist de buenas prácticas (transversal)

- **Seguridad:** HTTPS en producción; cookies [httpOnly](#) + [secure](#); nunca guardar tokens en LocalStorage; hashing de contraseñas; rate limiting; CORS configurado.
- **Datos:** validar entradas (Zod/Joi); índices en consultas frecuentes; paginación en listados grandes.
- **Código:** tipos compartidos backend/frontend (TypeScript); un componente = una responsabilidad; variables de entorno (no hardcodear secretos).
- **Rendimiento:** Server Components cuando sea posible; caché (Redis / [revalidate](#) / React Query-SWR); [next/image](#); lazy loading.
- **SEO:** metadata dinámica, JSON-LD, sitemap dinámico, robots.txt; medir con Lighthouse/PageSpeed/Search Console.

Índice de correcciones respecto al resumen anterior

El resumen previo ([resumen_curso.pdf](#)) se generó **solo con texto extraído** ([pdftotext](#)), por lo que perdió todo el contenido incrustado en imágenes: capturas de editor con código (fondo oscuro), diagramas y tablas renderizadas como imagen.

Metodología de detección: cada una de las 324 páginas se convirtió a JPG (150 dpi) y se analizó con dos señales automáticas — fracción de píxeles oscuros (detecta editores de código) y OCR (Tesseract) comparado contra el texto plano — para localizar páginas con contenido no capturado. Las páginas marcadas se revisaron **visualmente** y su código se transcribió a mano.

La siguiente tabla lista las páginas cuyo contenido **no estaba** en el texto plano y que aquí se recuperaron:

Sesión	Páginas recuperadas	Tipo de contenido recuperado
S01 — Node.js	p11, p19, p20, p21	Diagrama Event Loop; comandos Docker/instalación; código servidor HTTP nativo; diagrama de versiones LTS
S02 — Módulos/HTTP	p13, p14, p15, p16, p17	Servidor HTTP con enrutamiento manual; capturas Postman (GET/POST/PUT/DELETE con JSON reales)
S03 — Archivos/Streams	p09	Diagrama de pipes (código de streams ya estaba en el texto)
S04 — Express.js	p07, p08, p09, p10, p11, p13, p14, p15, p16, p18	Rutas básicas, <code>:id</code> param, <code>Router</code> modular, middlewares (logger y errores), diagramas, comandos Express Generator, árbol de archivos
S05 — APIs RESTful	p11, p12, p13, p14	CRUD completo (server.js, POST, GET, PUT, DELETE)
S06 — Base de datos	p12, p13, p14, p15, p16	Config y modelo Sequelize; <code>npm install mongoose</code> ; config y modelo Mongoose
S07 — Auth (backend)	p12, p14	Diagrama de flujo JWT; diagrama de arquitectura con middlewares (sin código de editor)
S08 — Despliegue	p10, p11, p14	Archivo <code>.env</code> ; captura de dashboard de logs; sección GitHub Actions
S09 — Intro Next.js	p11, p13, p14, p16	Diagramas Event/React 19; comando <code>create-next-app</code> (corregido <code>npm run dev</code>)
S10 — Enrutamiento	—	Sin código perdido: las páginas oscuras (p04, p13) eran fondos decorativos; el contenido estaba en texto
S11 — Componentes	p06, p07, p08, p09, p10, p11, p15	Anatomía completa de un componente React (11 partes) + eventos/patrones de props
S12 — API's Integradas	p06, p08, p09, p10, p11, p12	Route Handlers, GET/POST con repositorio, caché/cookies, CORS, path params, query params
S13 — Auth (NextAuth)	p06, p07, p08, p09	Config <code>CredentialsProvider</code> , <code>signIn</code> , sesión con <code>auth()</code> , protección de rutas
S14 — Optimización/SEO	p05, p06, p07, p09, p10, p11, p12, p13	ISR, fonts locales, <code>next/image</code> , <code>generateMetadata</code> , JSON-LD, sitemap XML/dinámico, robots.txt
S15 — Backend-Frontend	p06, p07, p08, p09, p11, p14	fetch, axios, tipos compartidos, setup Express, manejo de errores, validación Zod

Notas de precisión

Se transcribieron **fielmente** las capturas, incluyendo erratas tipográficas de las diapositivas originales, señaladas donde correspondía:

- **S06 p15:** la diapositiva muestra `console.erro("Error de conexión")` (doble errata); aquí corregido a `console.error("Error de conexión")`.
- **S09 p16:** la diapositiva muestra `npm rundev`; el comando correcto es `npm run dev`.
- **S13 (validaciones):** la diapositiva mostraba un ejemplo de clave segura poco legible; se documentó la práctica (`openssl rand -base64 32`) sin inventar el valor exacto.

No se identificó ningún fragmento de código ilegible que obligara a inventar contenido. Cuando un valor no era claro, se indicó explícitamente en lugar de suponerlo.